In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    ConfusionMatrixDisplay
)
```

In [2]:
```
df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")

print("Dataset shape:", df.shape)
print(df.head())
print(df.info())
```

```
Dataset shape: (7043, 21)
   customerID  gender  SeniorCitizen Partner Dependents  tenure PhoneService  \
0  7590-VHVEG  Female              0     Yes         No       1           No
1  5575-GNVDE    Male              0      No         No      34          Yes
2  3668-QPYBK    Male              0      No         No       2          Yes
3  7795-CFOCW    Male              0      No         No      45           No
4  9237-HQITU  Female              0      No         No       2          Yes

      MultipleLines InternetService OnlineSecurity  ... DeviceProtection  \
0  No phone service             DSL             No  ...               No
1                No             DSL            Yes  ...              Yes
2                No             DSL            Yes  ...               No
3  No phone service             DSL            Yes  ...              Yes
4                No     Fiber optic             No  ...               No

  TechSupport StreamingTV StreamingMovies        Contract PaperlessBilling  \
0          No          No              No  Month-to-month              Yes
1          No          No              No        One year               No
2          No          No              No  Month-to-month              Yes
3         Yes          No              No        One year               No
4          No          No              No  Month-to-month              Yes

               PaymentMethod MonthlyCharges  TotalCharges Churn
0           Electronic check          29.85         29.85    No
1               Mailed check          56.95        1889.5    No
2               Mailed check          53.85        108.15   Yes
3  Bank transfer (automatic)          42.30       1840.75    No
4           Electronic check          70.70        151.65   Yes

[5 rows x 21 columns]
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
None
```

In [3]:
```python
print(df.describe(include="all").T)
print("\nChurn distribution:")
print(df["Churn"].value_counts())
print("\nChurn percentage:")
print(df["Churn"].value_counts(normalize=True) * 100)
```

```
                  count unique              top  freq       mean        std  \
customerID         7043   7043       7590-VHVEG     1        NaN        NaN
gender             7043      2             Male  3555        NaN        NaN
SeniorCitizen    7043.0    NaN              NaN   NaN   0.162147   0.368612
Partner            7043      2               No  3641        NaN        NaN
Dependents         7043      2               No  4933        NaN        NaN
tenure           7043.0    NaN              NaN   NaN  32.371149  24.559481
PhoneService       7043      2              Yes  6361        NaN        NaN
MultipleLines      7043      3               No  3390        NaN        NaN
InternetService    7043      3      Fiber optic  3096        NaN        NaN
OnlineSecurity     7043      3               No  3498        NaN        NaN
OnlineBackup       7043      3               No  3088        NaN        NaN
DeviceProtection   7043      3               No  3095        NaN        NaN
TechSupport        7043      3               No  3473        NaN        NaN
StreamingTV        7043      3               No  2810        NaN        NaN
StreamingMovies    7043      3               No  2785        NaN        NaN
Contract           7043      3   Month-to-month  3875        NaN        NaN
PaperlessBilling   7043      2              Yes  4171        NaN        NaN
PaymentMethod      7043      4 Electronic check  2365        NaN        NaN
MonthlyCharges   7043.0    NaN              NaN   NaN  64.761692  30.090047
TotalCharges       7043   6531                    11        NaN        NaN
Churn              7043      2               No  5174        NaN        NaN

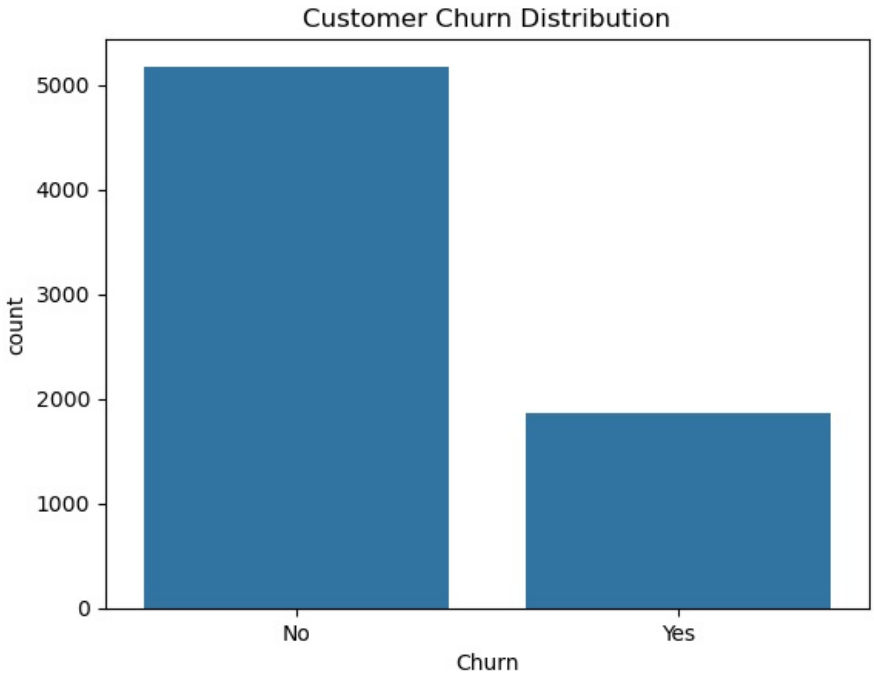
                   min   25%    50%    75%     max
customerID         NaN   NaN    NaN    NaN     NaN
gender             NaN   NaN    NaN    NaN     NaN
SeniorCitizen      0.0   0.0    0.0    0.0     1.0
Partner            NaN   NaN    NaN    NaN     NaN
Dependents         NaN   NaN    NaN    NaN     NaN
tenure             0.0   9.0   29.0   55.0    72.0
PhoneService       NaN   NaN    NaN    NaN     NaN
MultipleLines      NaN   NaN    NaN    NaN     NaN
InternetService    NaN   NaN    NaN    NaN     NaN
OnlineSecurity     NaN   NaN    NaN    NaN     NaN
OnlineBackup       NaN   NaN    NaN    NaN     NaN
DeviceProtection   NaN   NaN    NaN    NaN     NaN
TechSupport        NaN   NaN    NaN    NaN     NaN
StreamingTV        NaN   NaN    NaN    NaN     NaN
StreamingMovies    NaN   NaN    NaN    NaN     NaN
Contract           NaN   NaN    NaN    NaN     NaN
PaperlessBilling   NaN   NaN    NaN    NaN     NaN
PaymentMethod      NaN   NaN    NaN    NaN     NaN
MonthlyCharges   18.25  35.5  70.35  89.85  118.75
TotalCharges       NaN   NaN    NaN    NaN     NaN
Churn              NaN   NaN    NaN    NaN     NaN

Churn distribution:
Churn
No     5174
Yes    1869
Name: count, dtype: int64

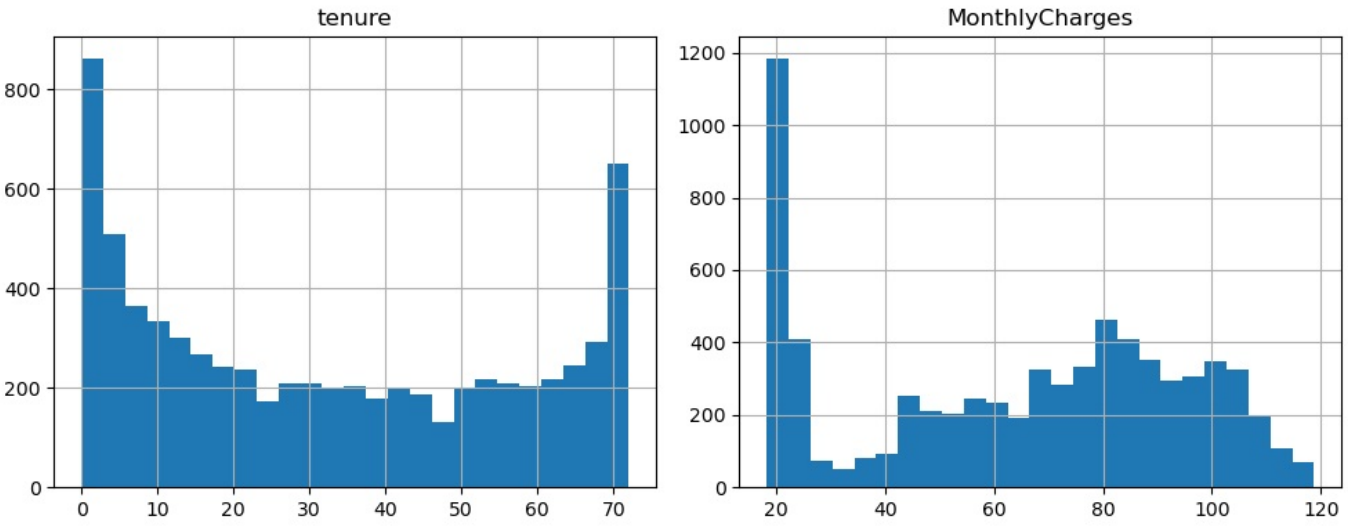
Churn percentage:
Churn
No     73.463013
Yes    26.536987
Name: proportion, dtype: float64
```

In [4]:

```
sns.countplot(data=df, x="Churn")
plt.title("Customer Churn Distribution")
plt.show()
```

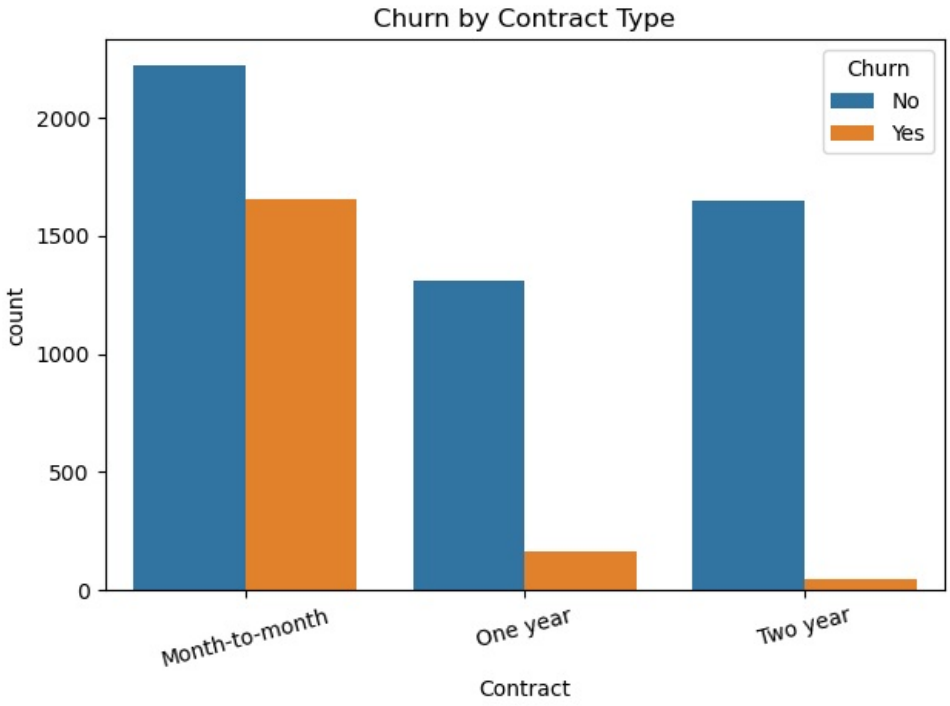


In [5]:
```
df[["tenure", "MonthlyCharges"]].hist(
    figsize=(10, 4), bins=25
)
plt.tight_layout()
plt.show()
```

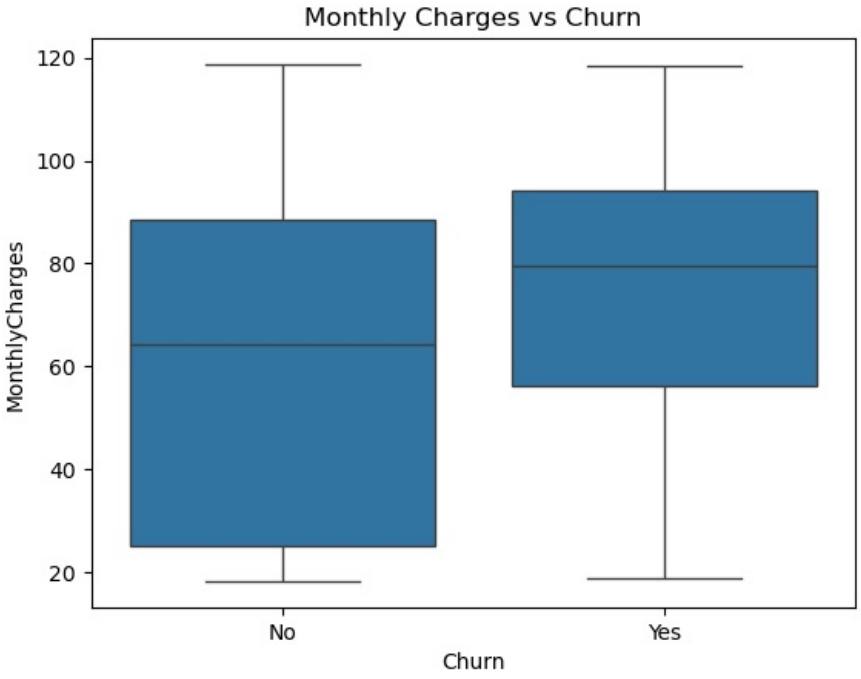


In [ ]:
```
```

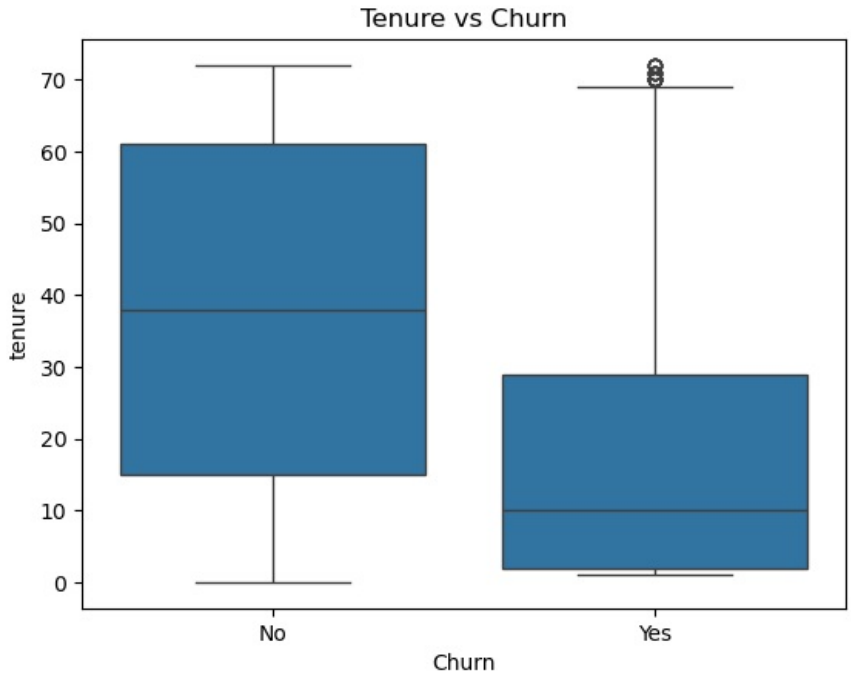
In [6]:
```
sns.countplot(data=df, x="Contract", hue="Churn")
plt.title("Churn by Contract Type")
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()
```



In [7]:
```
sns.boxplot(data=df, x="Churn", y="MonthlyCharges")
plt.title("Monthly Charges vs Churn")
plt.show()
```



In [8]:
```
sns.boxplot(data=df, x="Churn", y="tenure")
plt.title("Tenure vs Churn")
plt.show()
```



In [9]:
```
print("Missing values before cleaning:")
print(df.isnull().sum())
```

```
Missing values before cleaning:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
```

In [10]:
```
print("Duplicate rows:", df.duplicated().sum())
```

```
Duplicate rows: 0
```

In [11]:
```
print("\nData types:")
print(df.dtypes)
```

```
Data types:
customerID          object
gender              object
SeniorCitizen        int64
Partner             object
Dependents          object
tenure               int64
PhoneService        object
MultipleLines       object
InternetService     object
OnlineSecurity      object
OnlineBackup        object
DeviceProtection    object
TechSupport         object
StreamingTV         object
StreamingMovies     object
Contract            object
PaperlessBilling    object
PaymentMethod       object
MonthlyCharges     float64
TotalCharges        object
Churn               object
dtype: object
```

In [12]:
```
df = df.drop(columns=["customerID"], errors="ignore")
```

In [13]:
```
df["TotalCharges"] = pd.to_numeric(
    df["TotalCharges"], errors="coerce"
)
print("\nMissing values after conversion:")
print(df.isnull().sum())
df["TotalCharges"] = df["TotalCharges"].fillna(
    df["TotalCharges"].median()
)
df = df.drop_duplicates()
print("\nFinal shape after cleaning:", df.shape)
print("Remaining missing values:", df.isnull().sum().sum())
```

```
Missing values after conversion:
gender               0
SeniorCitizen        0
Partner              0
Dependents           0
tenure               0
PhoneService         0
MultipleLines        0
InternetService      0
OnlineSecurity       0
OnlineBackup         0
DeviceProtection     0
TechSupport          0
StreamingTV          0
StreamingMovies      0
Contract             0
PaperlessBilling     0
PaymentMethod        0
MonthlyCharges       0
TotalCharges        11
Churn                0
dtype: int64

Final shape after cleaning: (7021, 20)
Remaining missing values: 0
```

In [14]:
```
numeric_cols = [
    "tenure",
    "MonthlyCharges",
    "TotalCharges"
]
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

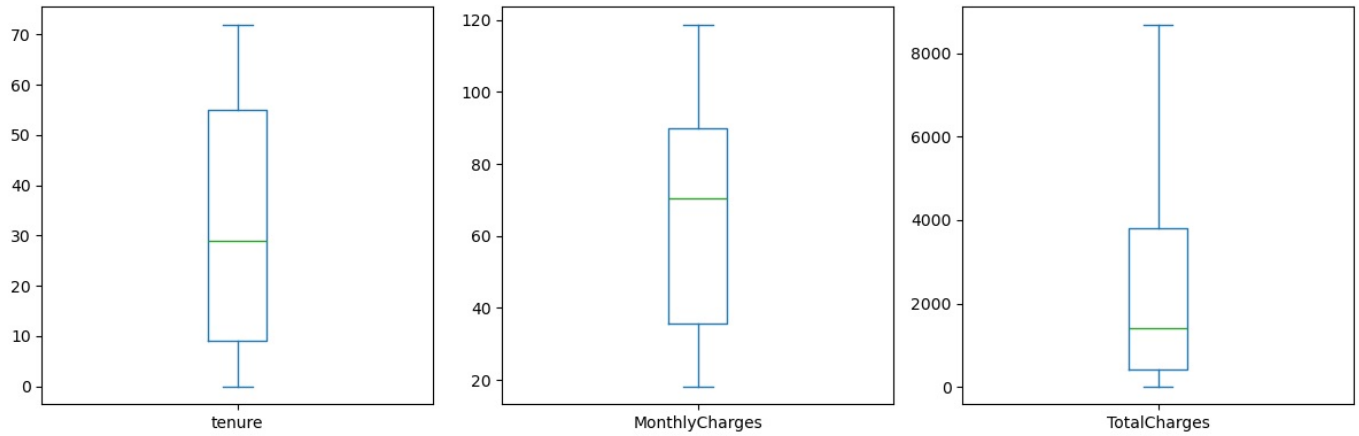
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    count = (
        (df[col] < lower) |
        (df[col] > upper)
    ).sum()

    print(f"{col}: {count} potential outliers")
```

```
tenure: 0 potential outliers
MonthlyCharges: 0 potential outliers
TotalCharges: 0 potential outliers
```

In [15]:
```
df[numeric_cols].plot(
    kind="box", subplots=True,
    layout=(1, 3), figsize=(12, 4)
)
plt.tight_layout()
plt.show()
```



In [16]:
```
X = df.drop(columns=["Churn"])
y = df["Churn"].map({"No": 0, "Yes": 1})
numeric_features = X.select_dtypes(
    include=["number"]
).columns.tolist()

categorical_features = X.select_dtypes(
    include=["object", "category", "bool"]
).columns.tolist()
numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])
categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])
preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features)
    ]
)
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
print("Training features before encoding:", X_train.shape[1])
```

```
Training samples: 5616
Testing samples: 1405
Training features before encoding: 19
```

In [19]:
```
models = {
    "Logistic Regression": LogisticRegression(
        max_iter=2000,
        class_weight="balanced",
        random_state=42
    ),

    "Decision Tree": DecisionTreeClassifier(
        class_weight="balanced",
        random_state=42
    ),

    "Random Forest": RandomForestClassifier(
        n_estimators=200,
        class_weight="balanced",
        random_state=42,
        n_jobs=-1
    )
}
trained_models = {}
results = []
predictions = {}
for name, model in models.items():

    pipeline = Pipeline(steps=[
        ("preprocessor", preprocessor),
        ("classifier", model)
    ])
```

```
    pipeline.fit(X_train, y_train)

    y_pred = pipeline.predict(X_test)

    trained_models[name] = pipeline
    predictions[name] = y_pred

    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(
            y_test, y_pred, zero_division=0
        ),
        "Recall": recall_score(
            y_test, y_pred, zero_division=0
        ),
        "F1-score": f1_score(
            y_test, y_pred, zero_division=0
        )
    })
results_df = pd.DataFrame(results)
print(results_df.round(4).to_string(index=False))
```

```
              Model  Accuracy  Precision  Recall  F1-score
Logistic Regression    0.7402     0.5061  0.7769    0.6129
      Decision Tree    0.7445     0.5176  0.5134    0.5155
      Random Forest    0.7701     0.5904  0.4301    0.4977
```

In [20]:

```
for name, y_pred in predictions.items():

    print("\n", "=" * 50)
    print(name)
    print("=" * 50)

    print(classification_report(
        y_test,
        y_pred,
        target_names=["No Churn", "Churn"],
        zero_division=0
    ))

    cm = confusion_matrix(y_test, y_pred)

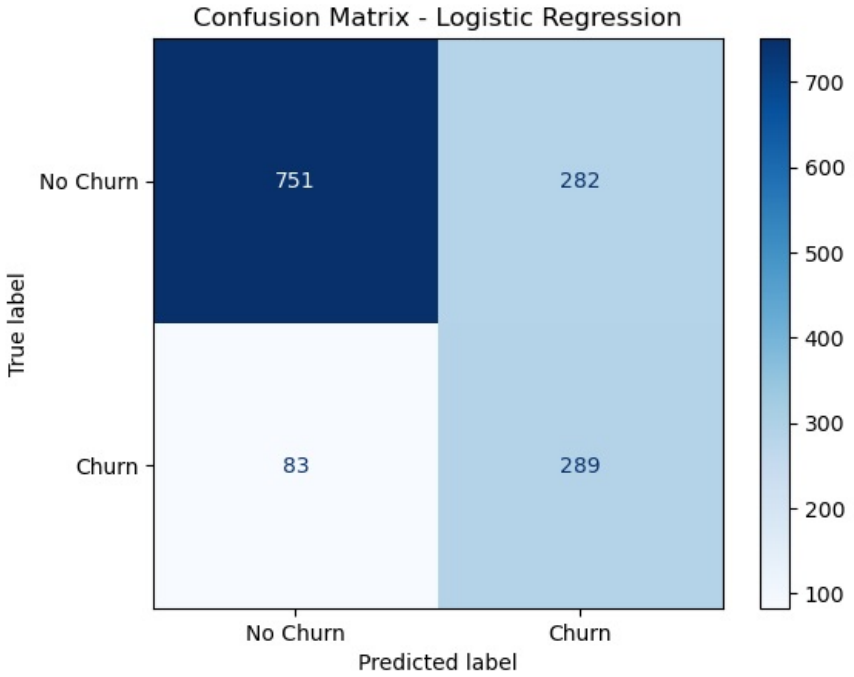
    ConfusionMatrixDisplay(
        confusion_matrix=cm,
        display_labels=["No Churn", "Churn"]
    ).plot(cmap="Blues", values_format="d")

    plt.title(f"Confusion Matrix - {name}")
    plt.show()
```

```
 ==================================================
Logistic Regression
==================================================
              precision    recall  f1-score   support

    No Churn       0.90      0.73      0.80      1033
       Churn       0.51      0.78      0.61       372

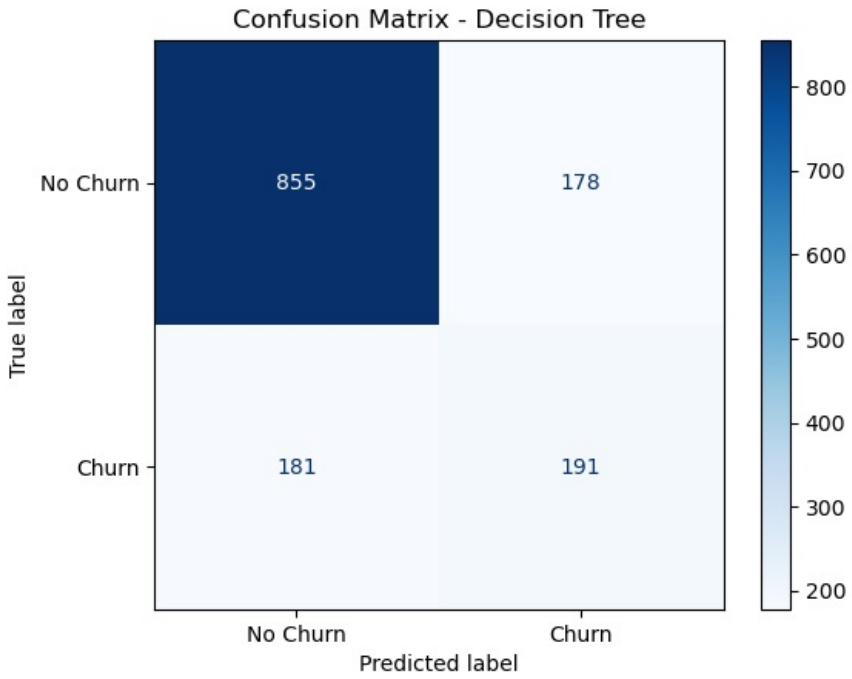
    accuracy                           0.74      1405
   macro avg       0.70      0.75      0.71      1405
weighted avg       0.80      0.74      0.75      1405
```




```
 ================================================
Decision Tree
================================================
              precision    recall  f1-score   support

    No Churn       0.83      0.83      0.83      1033
       Churn       0.52      0.51      0.52       372

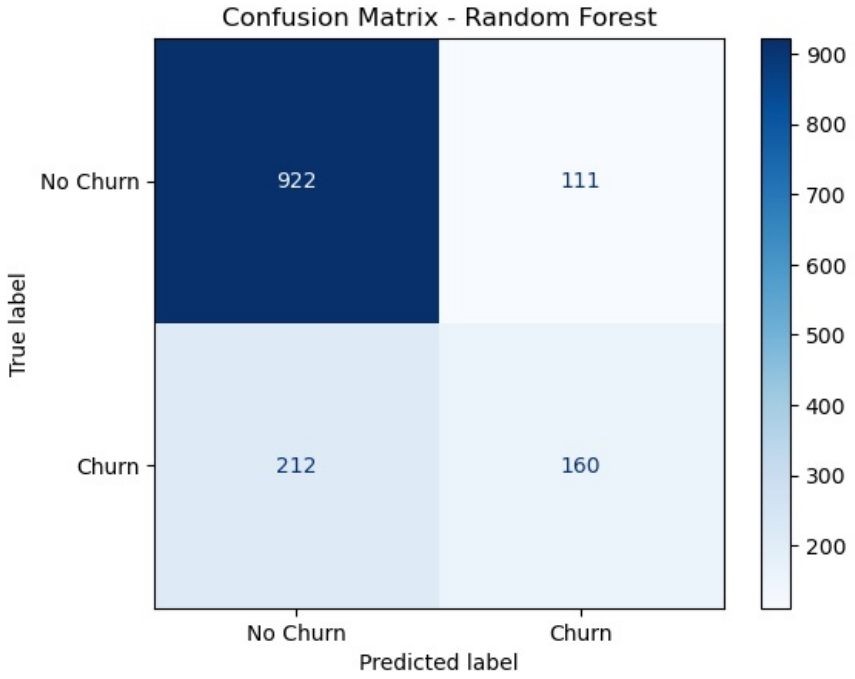
    accuracy                           0.74      1405
   macro avg       0.67      0.67      0.67      1405
weighted avg       0.74      0.74      0.74      1405
```




```
 ================================================
Random Forest
================================================
              precision    recall  f1-score   support

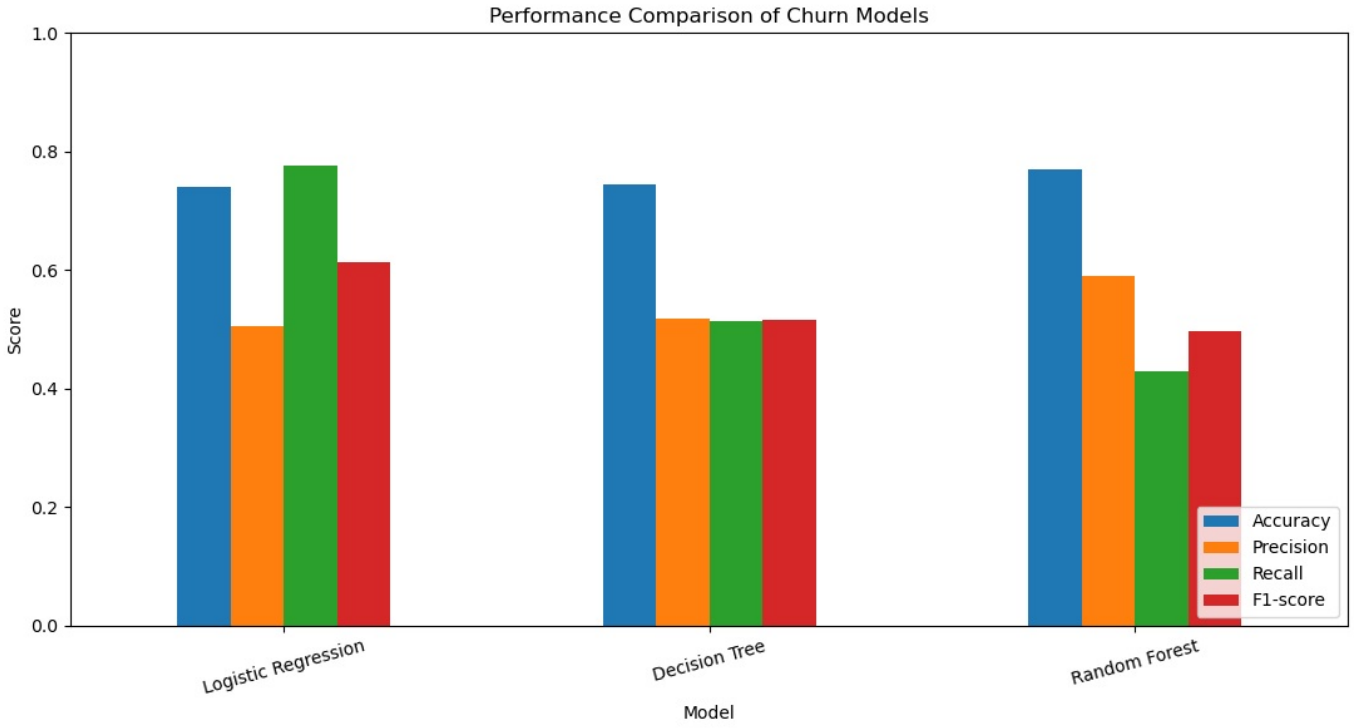
    No Churn       0.81      0.89      0.85      1033
       Churn       0.59      0.43      0.50       372

    accuracy                           0.77      1405
   macro avg       0.70      0.66      0.67      1405
weighted avg       0.75      0.77      0.76      1405
```



In [21]:

```
metrics = ["Accuracy", "Precision", "Recall", "F1-score"]
results_df.set_index("Model")[metrics].plot(
    kind="bar",
    figsize=(11, 6),
    ylim=(0, 1)
)
plt.title("Performance Comparison of Churn Models")
plt.ylabel("Score")
plt.xticks(rotation=15)
plt.legend(loc="lower right")
plt.tight_layout()
plt.show()
print("\nModels ranked by F1-score:")
print(
    results_df.sort_values(
        "F1-score", ascending=False
    ).round(4).to_string(index=False)
)
```



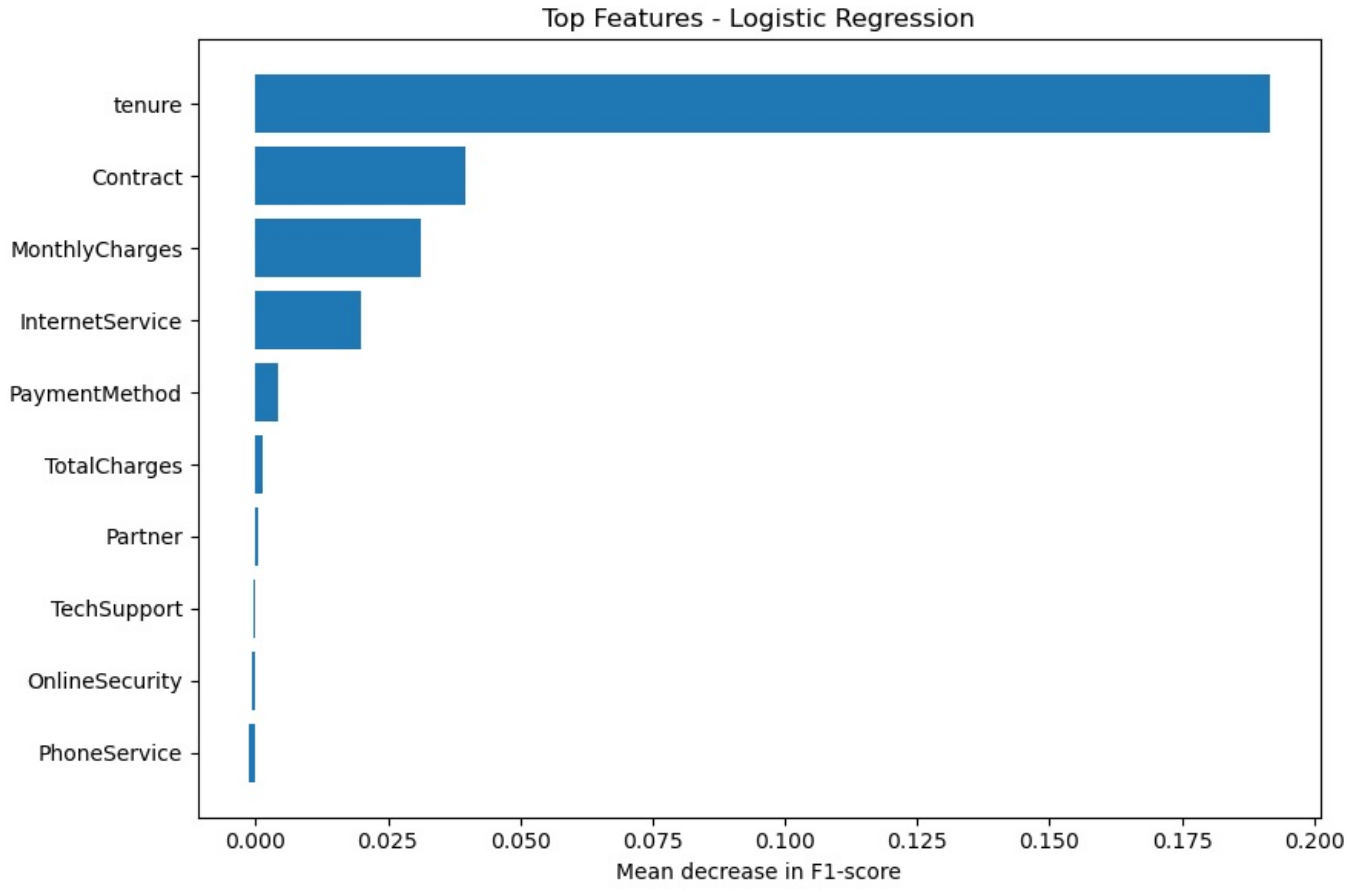

```
Models ranked by F1-score:
              Model  Accuracy  Precision  Recall  F1-score
Logistic Regression    0.7402     0.5061  0.7769    0.6129
      Decision Tree    0.7445     0.5176  0.5134    0.5155
      Random Forest    0.7701     0.5904  0.4301    0.4977
```

In [22]:

```python
from sklearn.inspection import permutation_importance
best_name = results_df.loc[
    results_df["F1-score"].idxmax(), "Model"
]
best_model = trained_models[best_name]
importance = permutation_importance(
    best_model,
    X_test,
    y_test,
    scoring="f1",
    n_repeats=5,
    random_state=42,
    n_jobs=-1
)
importance_df = pd.DataFrame({
    "Feature": X_test.columns,
    "Importance": importance.importances_mean
}).sort_values("Importance", ascending=False)
print("Best model:", best_name)
print("\nTop 10 important features:")
print(importance_df.head(10).round(4).to_string(index=False))
top10 = importance_df.head(10).sort_values("Importance")
plt.figure(figsize=(9, 6))
plt.barh(top10["Feature"], top10["Importance"])
plt.xlabel("Mean decrease in F1-score")
plt.title(f"Top Features - {best_name}")
plt.tight_layout()
plt.show()
```

```
Best model: Logistic Regression

Top 10 important features:
        Feature  Importance
         tenure      0.1915
       Contract      0.0395
 MonthlyCharges      0.0312
InternetService      0.0199
  PaymentMethod      0.0044
   TotalCharges      0.0012
        Partner      0.0005
    TechSupport     -0.0004
 OnlineSecurity     -0.0008
   PhoneService     -0.0012
```


Top Features - Logistic Regression
tenure
Contract
MonthlyCharges
InternetService
PaymentMethod
TotalCharges
Partner
TechSupport
OnlineSecurity
PhoneService
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175
0.200
Mean decrease in F1-score

In [ ]: